

人工智能训练师 理论知识点讲解04

刘老师

2024-10



目录

CONTENTS

- 相关系数
- 概率抽样和非概率抽样
- 数据的集中趋势和离散程度
- 系统运维概念
- Docker简介
- 负载均衡
- 智能系统运维的挑战

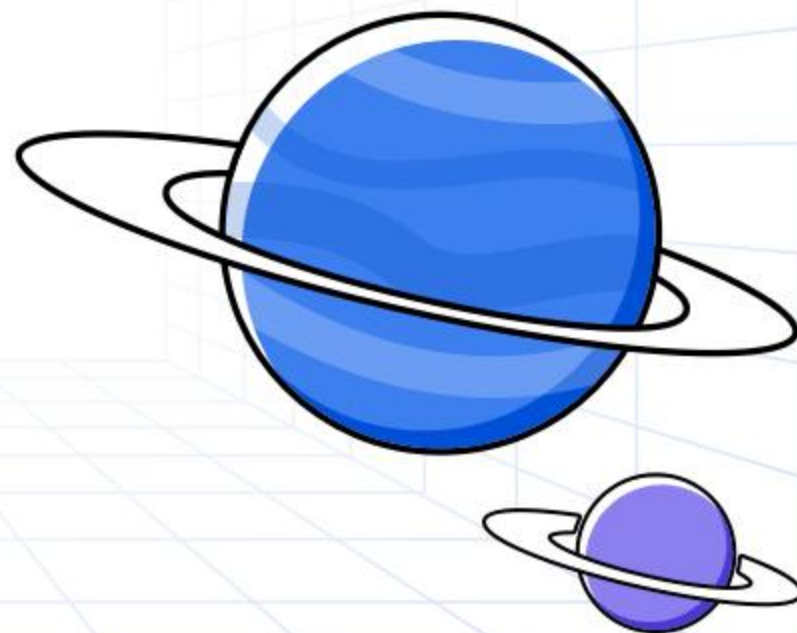
目录

CONTENTS

- 运维的阶段划分
- 数据库运维中的策略
- 系统中的性能分析
- 计算机开机自检过程
- 数据粒度
- 阻止网络数据采集的方式

01

相关系数



相关系数的主要作用

01

描述变量间关联程度

相关系数可帮助我们了解两个变量之间的线性相关紧密程度。

02

预测

基于已知的相关关系，根据一个变量的观测值，对另一个变量的值进行预测。

03

验证理论或假设

检验关于变量之间关系的理论或假设，确保理论或假设与实际数据相符。



相关系数的取值范围 $[-1,1]$

一个人的运动时间和体重变化之间的关系

相关系数为1时

01

表示两个变量之间存在完全正线性相关，即一个变量的增加会导致另一个变量按比例增加。

02

相关系数为-1时

表示两个变量之间存在完全负线性相关，即一个变量的增加会导致另一个变量按比例减少。

中文四六级

相关系数为0时

03

表示两个变量之间不存在线性相关关系，但可能存在其他非线性关系。

一个地区的降雨量和当地手机销量

04

相关系数的绝对值越接近1

则线性关系越强，越接近0，线性关系越弱。

常见的相关系数



皮尔逊相关系数

适用于两个变量均为数值型且大致呈线性关系的数据，计算简单且有效。

斯皮尔曼等级相关系数

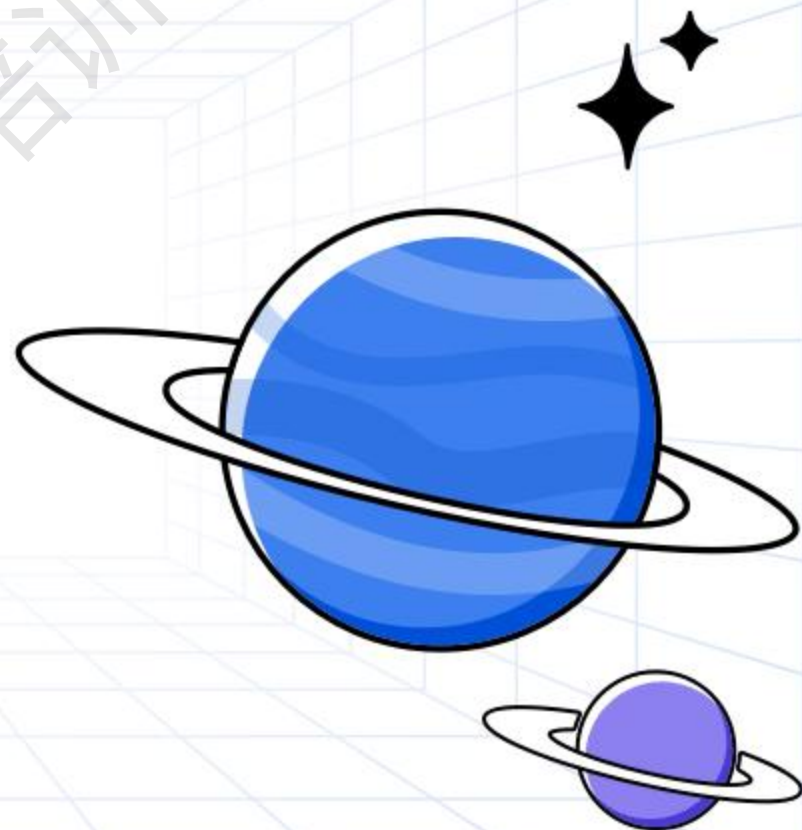
适用于两个变量均为等级数据或顺序数据的情况，可以衡量两个变量之间的单调关系。

肯德尔等级相关系数

适用于两个变量均为等级数据或顺序数据的情况，可以衡量两个变量之间的等级相关性。

02

概率抽样和非概率 抽样



概率抽样的优点

客观性

概率抽样能比较客观地反映总体情况，因为每个个体被抽中的概率是已知的，可以计算和控制抽样误差。

准确性

概率抽样的操作相对复杂，成本较高，但抽样误差可以计算和控制，确保了结果的**准确性和可靠性**。



英语四级考试

非概率抽样的优点



简便性

非概率抽样操作简便、成本低，因为不遵循随机原则，所以样本的代表性难以保证，也无法准确计算抽样误差。

快速性

非概率抽样适用于一些快速的、探索性的研究或者难以进行概率抽样的情况，能够快速获得样本并进行分析。

概率抽样的种类

简单随机抽样

从总体中随机抽取一定数量的个体，每个个体被抽取的概率相等。

系统抽样

将总体中的个体按一定顺序编号，然后按照固定的间隔抽取样本，又称等距抽样或机械抽样。



分层抽样

将总体按照某些特征分成不同的层次，然后从每个层次中独立进行抽样。

整群抽样

将总体划分为若干群，随机抽取部分群，对抽中群的全部个体进行调查。

非概率抽样的种类

一个配额抽样的例子：

假设要对一个城市居民的消费习惯进行调查。已知该城市居民按照收入水平分为高收入、中等收入和低收入三个群体，比例分别为 20%、60% 和 20%。

计划抽取 1000 个样本，那么按照配额抽样的方法：

高收入群体的配额为 $1000 \times 20\% = 200$ 人。调查人员可以在高收入群体常出现的场所，如高档住宅区、高端商场等地进行抽样，直到抽到 200 个高收入者为止。

中等收入群体的配额为 $1000 \times 60\% = 600$ 人。可以在普通住宅区、超市、商场等地抽取，凑够 600 个中等收入者。

低收入群体的配额为 $1000 \times 20\% = 200$ 人。在经济适用房小区、廉价市场等地抽取 200 个低收入者。



方便抽样

在街头随意选择路人进行调查，这种方式简单且成本低，但样本代表性难以保证。

判断抽样

由研究者根据经验、主观判断选择被调查对象，这种方式简便易行，但准确性难以保证。

配额抽样

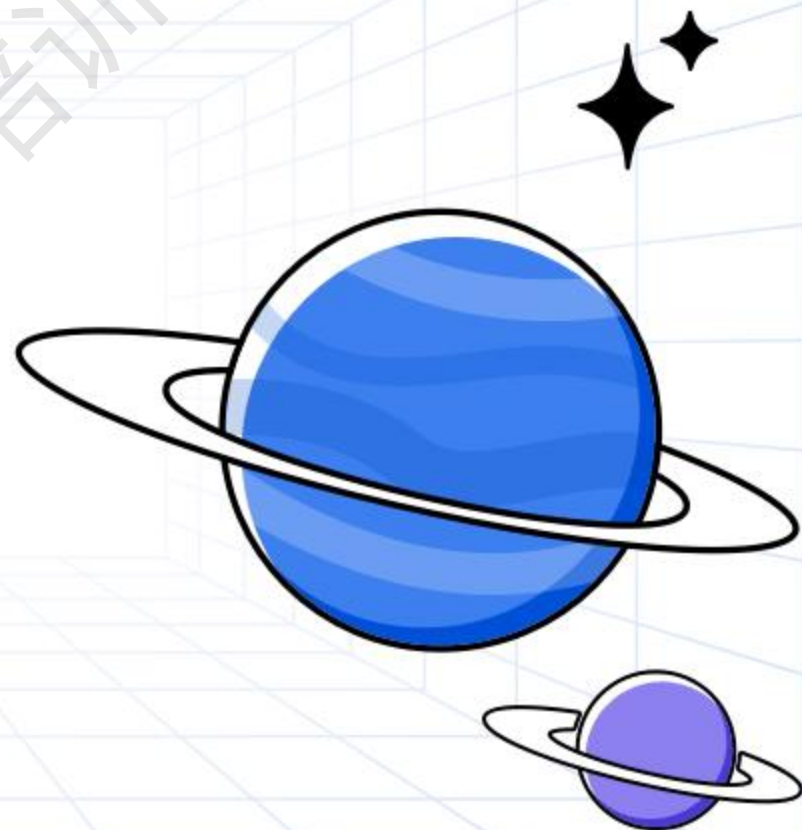
按照一定标准分配样本数额，然后由调查人员在规定的数额内选取样本，这种方式适用于快速调查。

滚雪球抽样

通过一些初始被调查者的推荐来获取更多的被调查者，这种方式适用于扩大样本规模。

03

数据的集中趋势和离散程度



数据的集中趋势



● 平均数

将所有成绩相加，然后除以30。总和约为2430，平均数约为81分，反映出班级整体的平均水平。描述性统计分析常用的指标中，**平均数容易受极值的影响**，当数据集中出现极值时结果将会出现较大的偏差。

● 中位数

将成绩从小到大排列，位于中间位置的数。排序后第15和第16个数分别是83和84，中位数为 $(83+84)/2=83.5$ 分。

● 众数

出现次数最多的数。在例子中，80、85和90出现的次数相对较多，都是众数。

数据的离散程度

方差:

$$s^2 = \frac{1}{n} [(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2]$$

标准差

$$s = \sqrt{\frac{1}{n} [(x_1 - \bar{x})^2 + (x_2 - \bar{x})^2 + \dots + (x_n - \bar{x})^2]}$$

极差

最大值98分减去最小值60分，极差为38分。极差反映了数据的最大差距。



方差

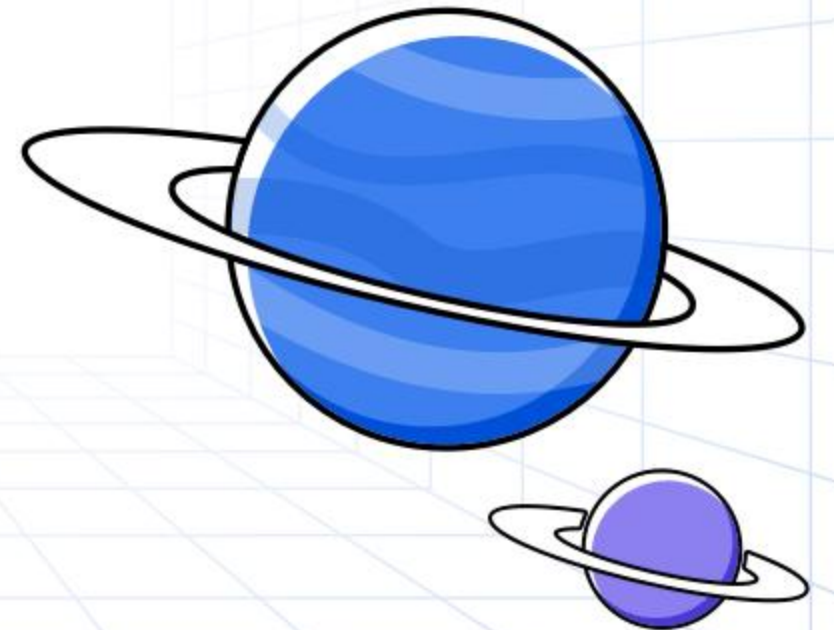
先计算每个数据与平均数的差值，将这些差值平方后求和，再除以数据个数。计算得出方差，它反映了数据相对于平均数的分散程度。

标准差

方差的平方根。标准差越大，数据的离散程度越大。

04

系统运维概念



系统运维概念



系统运维

系统运维通常指服务器、存储设备、网络设备等硬件的维护和管理，以及操作系统的安装、配置、优化和更新。



系统性能监测

系统运维的主要日常工作包括对系统性能的监测、故障的排查和处理、数据的备份和恢复、安全防护等。



早期阶段

在系统运维的发展历程中，早期阶段属于人工阶段，运维人员主要依靠手动操作来完成各项任务。

系统运维概念

硬件维护

早期系统运维确实更多侧重于硬件维护，因为硬件是系统运行的基础，当时的技术条件和业务需求使得保障硬件的正常运转成为首要任务。

威胁和挑战

随着技术的发展和业务的复杂多样化，系统面临的威胁和挑战不断增加，稳定性和安全性的重要性愈发凸显。

稳定性安全性

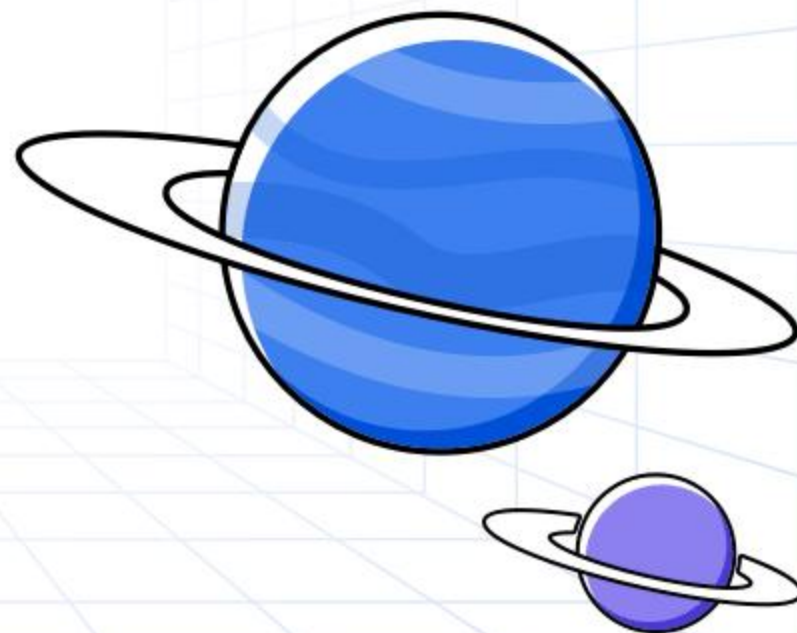
稳定性关乎系统能否持续、可靠地提供服务，避免出现故障或中断；安全性则涉及保护系统免受各种攻击、数据泄露等风险。

规范化标准化

同时，对运维流程的规范化、标准化，以及对运维数据的分析和利用也越来越重视。

05

Docker简介



Docker简介



Docker实现资源高效利用

Docker通过容器化技术、轻量级虚拟化和高效的资源分配，实现了资源的高效利用。

容器化技术

Docker将应用程序及其依赖项打包到一个独立的容器中，与宿主机和其他容器隔离，避免了应用之间的冲突。

轻量级虚拟化

Docker相比传统的虚拟机，容器更加轻量，启动速度快，占用资源少，它不需要完整地模拟硬件和操作系统。

Docker简介

高效的资源分配

Docker可以对容器进行资源限制和分配，如CPU、内存、网络带宽等，使得可以根据应用的实际需求精确地分配资源。

快速部署和迁移

Docker容器的部署速度快，并且可以在不同的环境中轻松迁移，使得资源能够快速根据业务需求进行调整和重新分配。

Docker搬家公司(类比)

Docker就像是一个超级高效的搬家公司，将复杂的家具组合打包成一个“包裹”，然后轻松地搬到任何地方。



Docker简介

资源利用效率

在计算机世界里，应用程序就像是那些家具，而Docker把应用程序及其所需的所有环境和依赖项打包成一个容器。

部署效率

Docker让应用程序可以在任何支持Docker的地方轻松运行，而且运行效果和原来的环境中一模一样。

Docker简介

一家小型创业公司正在开发一款新的 **Web** 应用程序。开发团队由前端开发人员、后端开发人员和运维人员组成。

在开发过程中，前端开发人员使用特定版本的 **Node.js** 来运行前端代码，后端开发人员则依赖特定版本的 **Python** 以及相关的库来运行后端服务。

当准备将应用程序部署到测试环境或生产环境时，**如果没有 Docker**，运维人员需要在服务器上分别安装和配置前端和后端所需的各种依赖和环境，这是一个繁琐且容易出错的过程。

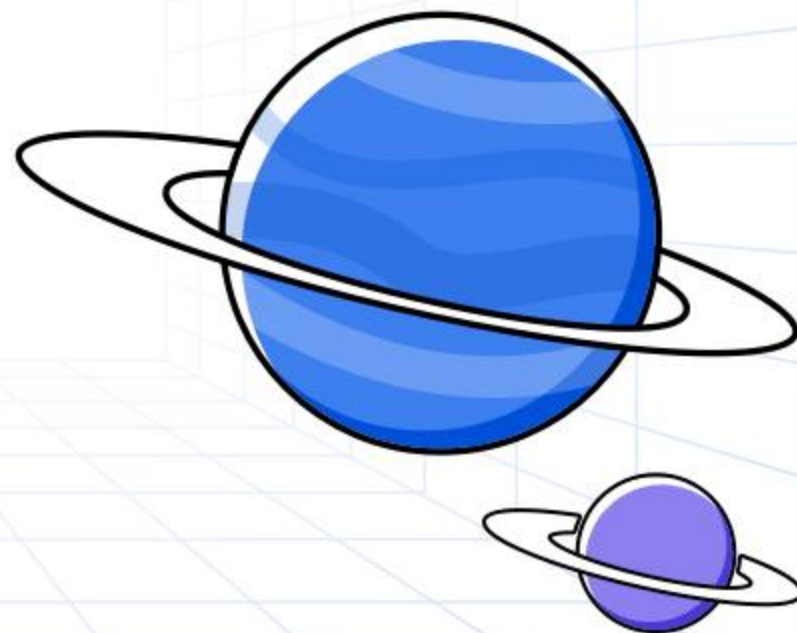
而**使用 Docker 后**，前端开发人员可以创建一个包含特定版本 **Node.js** 和前端项目代码的 **Docker** 镜像，并将相关配置写在 **Dockerfile** 中。后端开发人员同样创建包含特定版本 **Python** 及后端依赖的 **Docker** 镜像。

在部署时，运维人员只需要**拉取这些镜像**，然后运行相应的容器，无需关心复杂的环境配置问题，大大降低了部署的难度和出错的可能性，提高了部署的效率和一致性。

即使后续需要对应用程序进行升级或修改环境配置，只需要修改 **Dockerfile** 重新构建镜像，然后重新部署即可，不会对现有的运行环境造成影响。

06

负载均衡



负载均衡

负载均衡

负载均衡是将网络或计算任务均匀地分配到多个服务器或计算资源上，以提高系统的整体性能、可用性和可扩展性。

负载均衡的实现方式

负载均衡的实现方式多种多样，包括硬件负载均衡器和软件负载均衡器，以及各种不同的负载均衡算法。

01

02

负载均衡的作用

负载均衡的主要作用包括提高性能、增加可用性、实现可扩展性，以及确保系统的高效运行和快速响应。

03

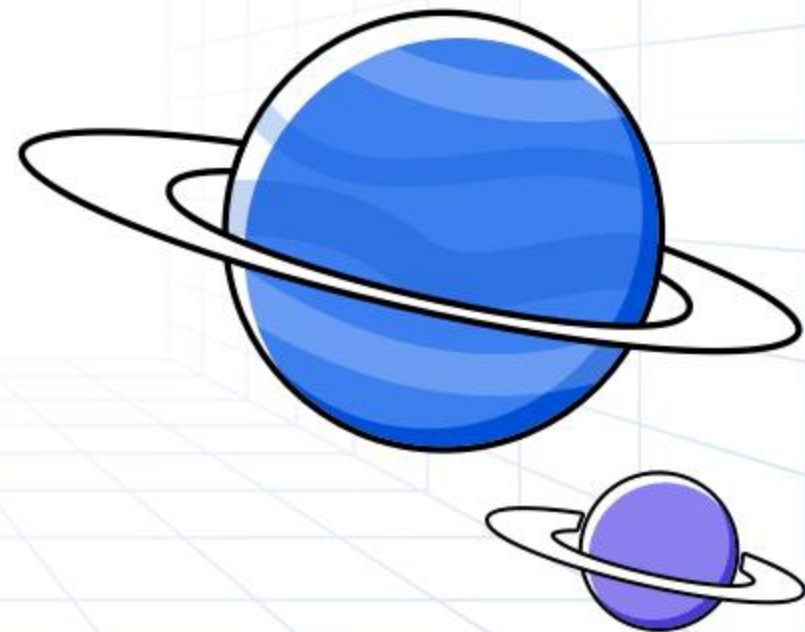
04

负载均衡算法

负载均衡算法有轮询、加权轮询、最少连接、源地址哈希等，根据不同的场景选择合适的算法实现最优的负载分配。

07

智能系统运维的挑战



智能系统运维的挑战

01

系统复杂度提高

随着系统复杂度的不断提高，运维工作难度加大，包括系统架构的理解、故障排查等。

02

场景日益丰富

场景日益丰富，需要适应多样化的业务需求和环境变化，增加了运维的复杂性。

03

用户流失严重

用户流失严重，需要分析原因并采取措施来挽留用户，提升用户体验。

智能系统运维的挑战

性能优化困难

响应时间、吞吐量等性能指标的提升面临挑战，难以预测的用户行为。

新技术快速更新

新技术快速更新，要持续跟进并应用新的算法、框架和工具。

法规和合规性要求严格

在数据处理和用户隐私保护方面需满足诸多规定。

智能系统运维的挑战

人才短缺

专业的智能系统运维人才匮乏，缺乏有效的故障诊断工具。



系统兼容性问题突出

不同模块、组件间及与外部系统的集成难度较大。



数据存储和处理成本高昂

大量数据的存储和处理投入成本高。



智能系统运维的挑战

难以预测的用户行为

用户行为模式多变，难以准确把握和应对。

对第三方服务依赖存在风险

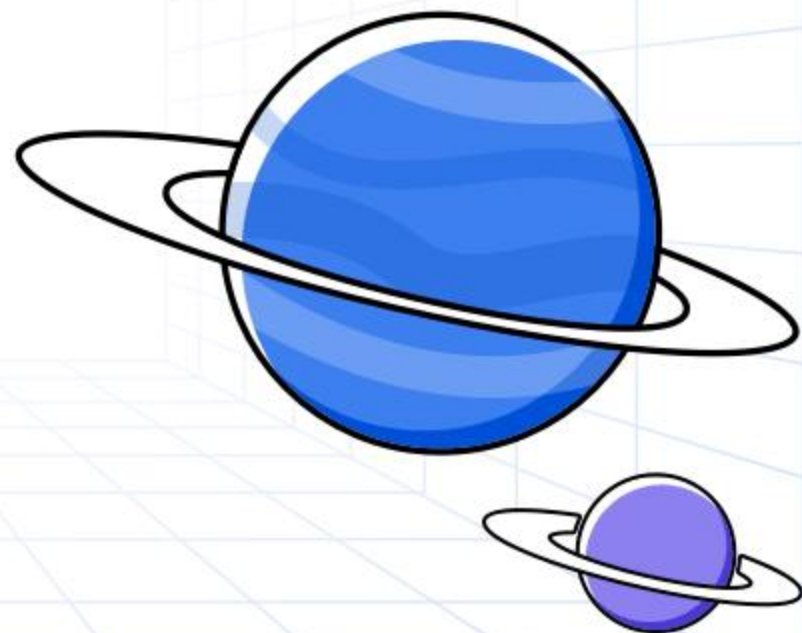
其稳定性和可用性难以完全掌控。

应急响应能力不足

面对突发状况缺乏有效的应对策略和流程。

08

运维的阶段划分



运维的阶段划分

人工运维阶段

运维工作主要依赖人工操作和处理，效率较低且容易出错。

平台化运维阶段

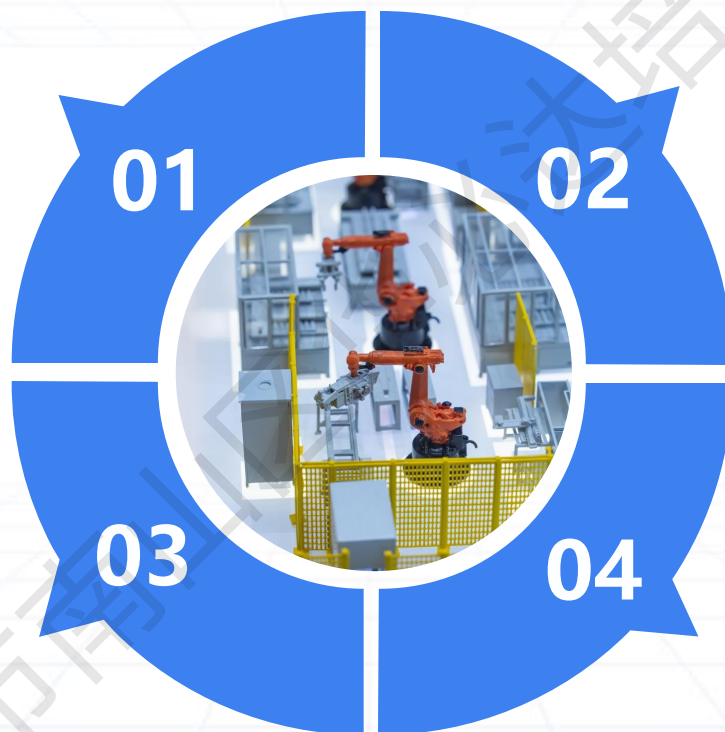
通过构建统一的运维平台，整合资源和流程，实现更高效的管理和协同。

工具与自动化运维阶段

借助各种工具和脚本实现部分重复性工作的自动化，提高了运维效率和准确性。

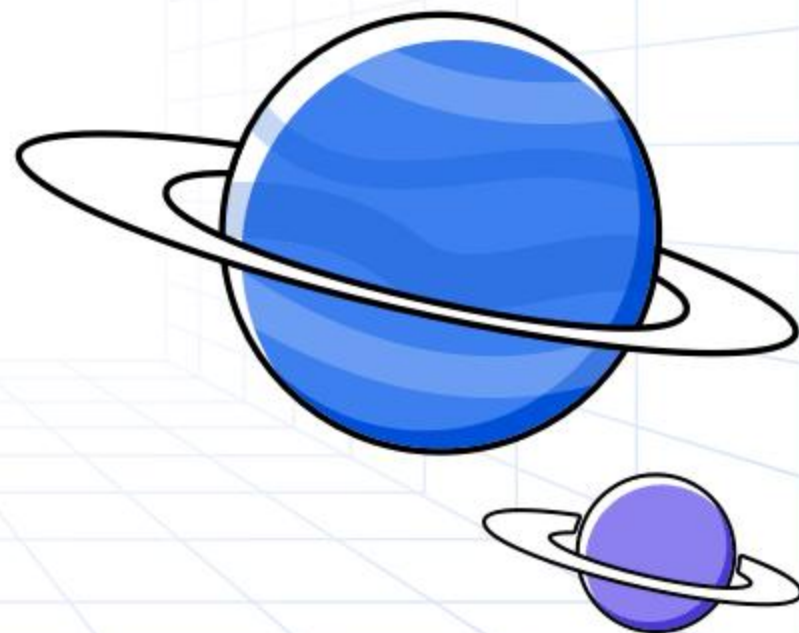
智能运维阶段

利用人工智能、大数据分析等技术，实现更智能的故障预测、资源优化等。

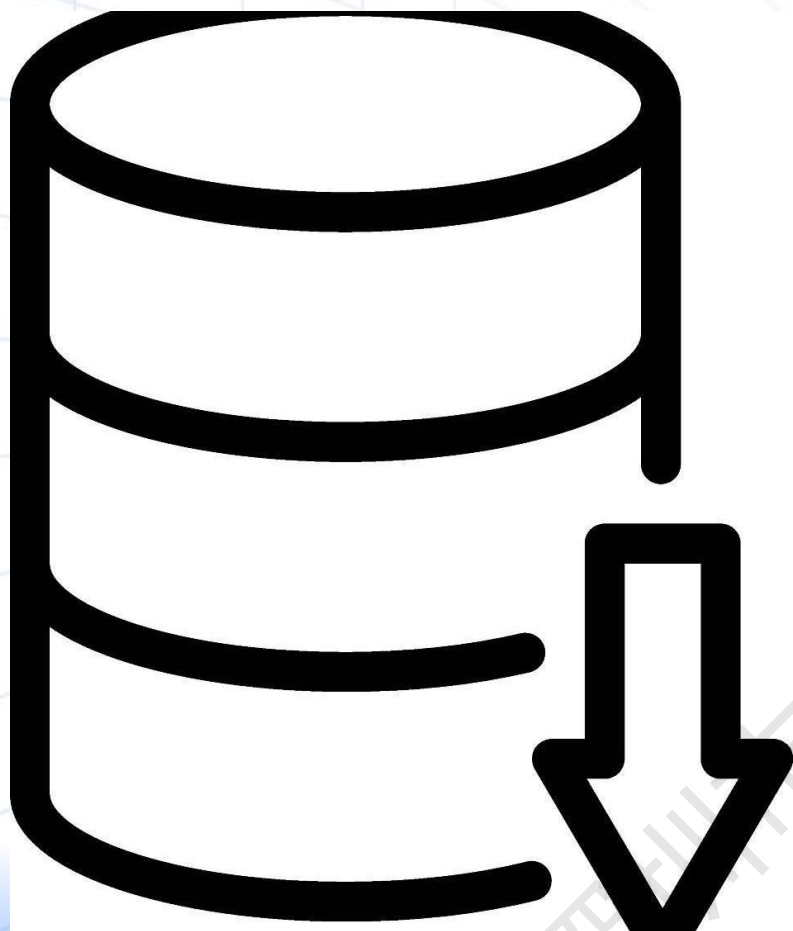


09

数据库运维中的策略



数据库运维中的策略



01

数据备份与恢复策略

缺乏数据备份和恢复策略可能导致数据丢失和完整性问题，包括硬件故障、软件错误、人为失误等。

02

硬件故障风险

硬件故障是常见的风险之一，服务器的硬盘可能会损坏、存储设备可能出现故障，如果没有备份，数据无法恢复。

03

软件错误

软件错误也时有发生，数据库系统可能会遭遇崩溃、错误的操作或配置更改，导致数据损坏或丢失。

数据库运维中的策略

人为失误

人为失误难以完全避免，例如误删除重要的数据表、错误地更新数据等操作，如果没有备份，这些失误将无法挽回。



恶意攻击

如果遭受恶意攻击，如病毒、黑客入侵等，数据可能被篡改、删除或加密，若没有有效的备份，恢复数据的完整性将极为困难。



不可抗力事件

自然灾害、火灾、洪水、电力故障等不可抗力事件也可能对数据库所在的物理环境造成破坏，导致数据丢失。



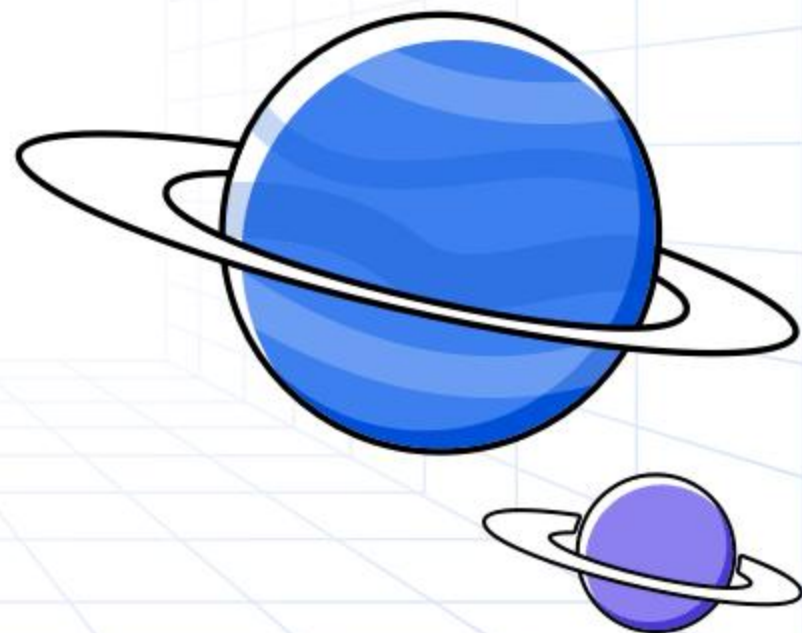
数据丢失与受损

缺乏数据备份和恢复策略会使数据库在面临各种风险和意外时，无法有效地保护数据，从而容易导致数据丢失和完整性受损。



10

系统管理中的性能 分析



系统管理的性能分析

CPU资源与网络性能

CPU资源和网络性能是保障系统运行效率和用户体验的关键，需密切关注。

应用程序性能

应用程序的性能直接影响到用户的工作效率和服务质量，响应时间和吞吐量等指标是评估应用程序运行状况的重要依据。

硬件管理角色

硬件管理在系统管理中的角色并不显著，因为硬件配置和维护工作在系统正常运行后变得相对稳定，不太可能成为性能瓶颈。

操作系统与系统软件

操作系统和底层系统软件通过对硬件的抽象和管理，为上层应用程序和性能分析提供了相对统一的接口和环境。

硬件故障与兼容性

硬件故障或不兼容等问题虽然相对较少发生，但一旦出现就会表现为明显的硬件故障，容易被单独排查和解决。

性能分析重点

性能分析的重点是找出可以通过软件优化、配置调整等方式来改进的方面，而硬件的升级或更换通常是成本较高且不太灵活的解决方案。

硬件管理在性能分析中的角色

操作系统和底层系统软件通过对硬件的抽象和管理，为上层应用程序和性能分析提供了相对统一的接口和环境。

性能分析的重点是找出可以通过软件优化、配置调整等方式来改进的方面，而硬件的升级或更换通常是成本较高且不太灵活的解决方案。

硬件管理稳定

抽象管理界面

硬件故障排查

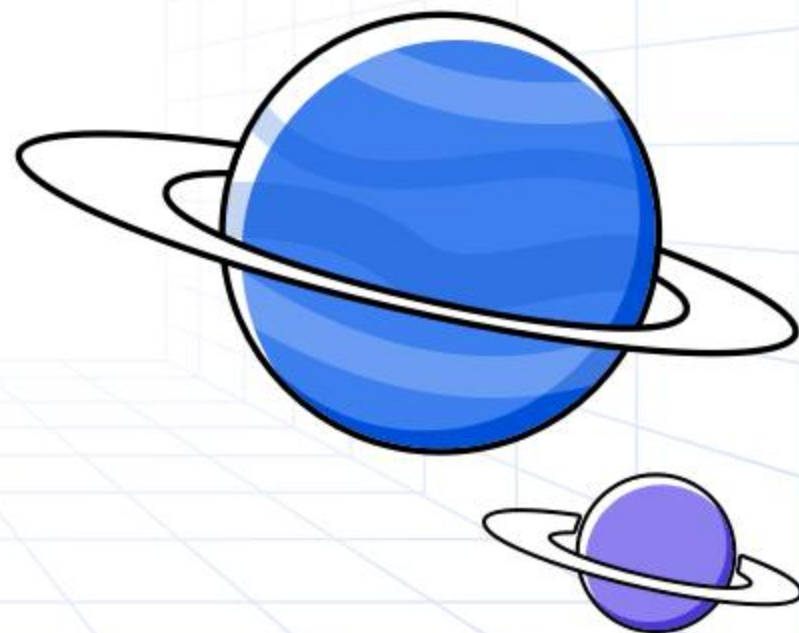
性能分析重点

硬件管理在系统管理中的角色并不显著，因为硬件配置和维护工作在系统正常运行后变得相对稳定，不太可能成为性能瓶颈。

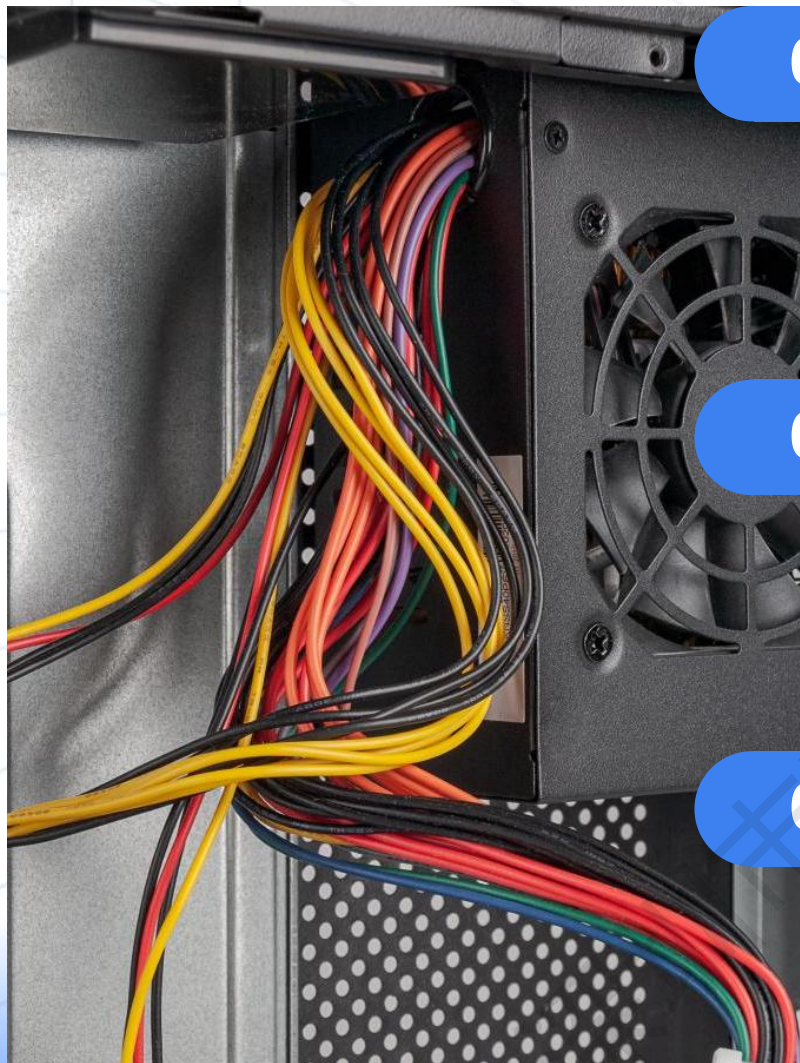
硬件故障或不兼容等问题虽然相对较少发生，但一旦出现就会表现为明显的硬件故障，**容易被单独排查和解决。**

11

计算机开机自检过程



计算机开机自检过程



01

电源接通

当按下电源按钮后，计算机电源开始向主板和其他组件供电。

02

硬件初始化

主板上的基本输入输出系统（BIOS）或统一可扩展固件接口（UEFI）被激活。

03

内存检测

检查系统内存（RAM）的存在和完整性，测试内存是否能够正常存储和读取数据。

计算机开机自检过程

04

显卡检测

确认显卡的存在和基本功能，以便能够在屏幕上显示信息。

05

其他硬件检测

包括硬盘、光驱、USB 接口、声卡、网卡等设备的检测，检查它们是否连接正常并能正常工作。

06

时钟检测

检查计算机的时钟芯片，确保时间设置正确。

计算机开机自检过程

BIOS/UEFI 自身检查

验证 BIOS/UEFI 程序的完整性和正确性。

启动设备选择

根据 BIOS/UEFI 中的设置，确定从哪个设备（如硬盘、光驱、U 盘等）启动操作系统。

警报声错误

如果在自检过程中发现严重错误，计算机可能会发出警报声（如长鸣、短鸣等）或在屏幕上显示错误信息。

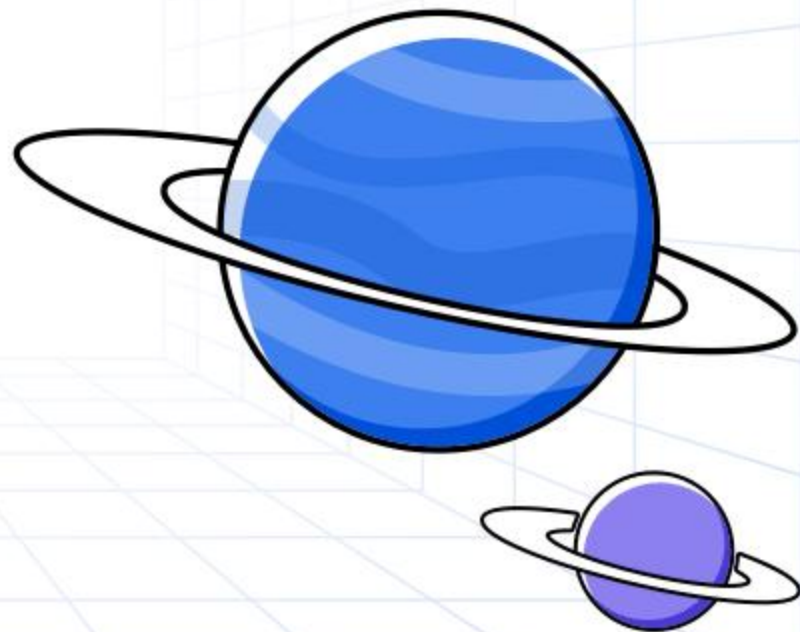
加载操作系统

如果自检通过，计算机将尝试从指定的启动设备加载操作系统。



12

数据粒度



数据粒度

数据粒度定义

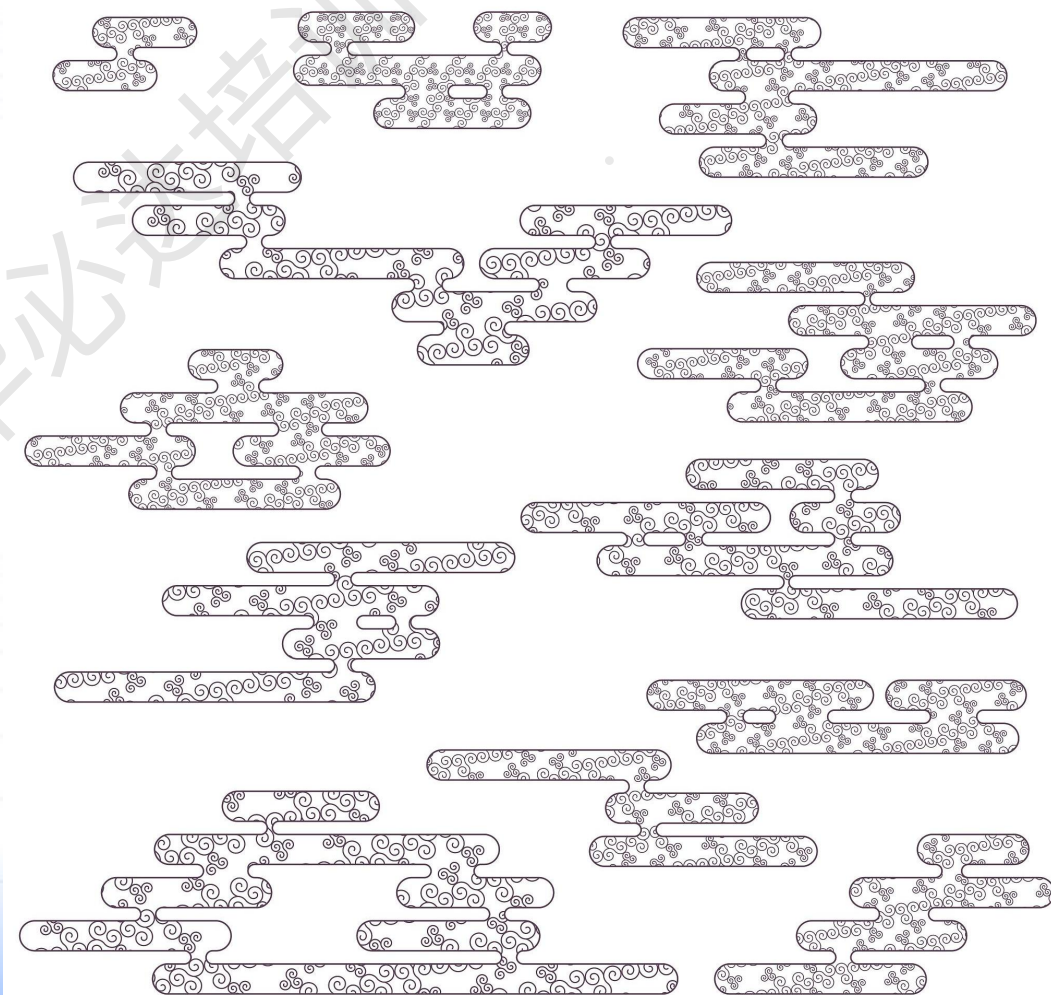
数据粒度是指数据仓库中数据的细化和综合程度，描述了数据的详细程度或汇总程度。

数据粒度选择

数据粒度的选择取决于数据的用途、存储和处理成本以及用户对数据详细程度的需求等因素。

粗粒度数据

粗粒度的数据是对细粒度数据进行汇总、概括得到的，例如一个月内所有交易的总金额。



数据粒度

细粒度数据

细粒度的数据包含更详细、更具体的信息，例如每一笔交易的详细记录。

学生成绩例子

记录班级学生的成绩，粗粒度只记录平均分，细粒度则详细记录每个学生每门学科的具体分数。

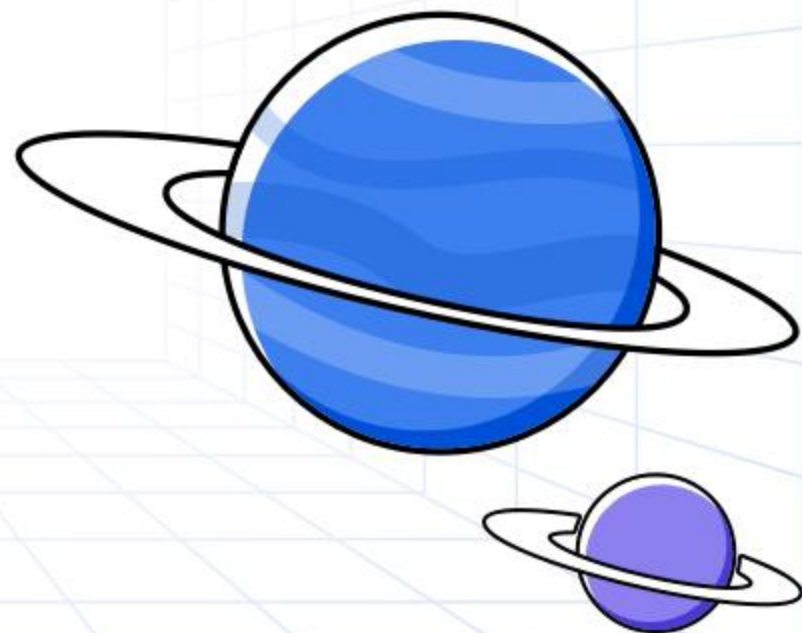
水果记录例子

想象有一堆水果，记录它们的信息，粗粒度只记录总数和总重量，细粒度则详细记录每个水果的信息。



13

阻止网络数据采集的方式



阻止网络数据采集的方式

验证码

在网页中显示需要用户识别和输入的验证码，只有正确输入才能继续访问或执行操作。

蜜罐字段

在表单中添加一些隐藏的字段，正常用户在填写表单时不会触及这些字段，但网络爬虫可能会填写。

陷阱区分

通过设置特殊的、不易被正常用户察觉但爬虫可能会触发的“陷阱”来区分正常用户和爬虫。

验证码识别

验证码通常难以被爬虫准确识别和输入，因此能有效区分正常用户和爬虫。

蜜罐字段填写

利用爬虫自动填写表单的特性，正常用户不会填写隐藏字段，而爬虫可能会填写。

判断爬虫行为

通过判断填写隐藏字段的行为，可以判断出这不是正常的用户行为，从而识别出爬虫。



◆ THANKS ◆
感谢您的观看

